APUNTES GIT
GitHub Actions y GitHub Pages y Git Ignore

RESUMEN RAPIDO

1. Fase Local: Creación y Primeros Archivos (git init + .gitignore + commit)

Antes de subir nada a internet, preparas la casa en tu ordenador.

- **Concepto:** Creas el repositorio y decides qué archivos **no** quieres que se guarden (como contraseñas, dependencias, carpetas temporales).
- **Comandos clave y ejemplo:**
- Bash

```
# 1. Inicializar git en la carpeta
git init

# 2. Crear un archivo .gitignore para ignorar cosas (ej: node_modules o .env)
touch .gitignore
# (Dentro de .gitignore escribirías: node_modules/)

# 3. Añadir archivos al "escenario de preparación"
git add .

# 4. Guardar la foto oficial (Commit)
git commit -m "Primer commit: estructura inicial"
```

-
-

2. Fase de Ramas y Control de Errores (branch, reset y revert)

Trabajas en paralelo y aprendes a solucionar cagadas o viajar en el tiempo.

- **Concepto de Ramas:** Creas caminos paralelos (feature, dev, main) para no romper el código principal.
- **Concepto de Errores (reset vs revert):**
    - git reset: Borra commits hacia atrás (como si nunca hubieran pasado). ¡Cuidado si ya has hecho push!

    - git revert: Crea un commit nuevo que "deshace" los cambios del commit anterior de forma segura (ideal si ya lo has subido a GitHub).
- **Comandos clave y ejemplo:**
- Bash

```
# Crear y cambiarte a una rama nueva
git checkout -b mi-nueva-rama   # (o git switch -c mi-nueva-rama)

# Volver a main y fusionar
git checkout main
git merge mi-nueva-rama

# Viajar en el tiempo (Ejemplo de revert para anular un commit chungo)
git revert <codigo-del-commit>
```

-
-

3. Fase de Conexión a la Nube (git remote add + push)

Vinculas tu ordenador con GitHub por primera vez.

- **Concepto:** Construyes el puente entre tu ordenador y la nube. (Recuerda: esto solo se hace una vez por proyecto).
- **Comandos clave y ejemplo:**
- Bash

```
# Vincular el repositorio remoto
git remote add origin https://github.com/tu-usuario/tu-repositorio.git

# Subir tus ramas a GitHub por primera vez
git push -u origin main
```

-
-

4. Fase de Automatización (GitHub Actions y YAML)

Configuras tus robots para que trabajen por ti cuando haces push.

- **Concepto:** Creas una carpeta oculta con un archivo .yml que le dice a GitHub qué hacer automáticamente (pasar tests, compilar, etc.).
- **Estructura y pasos:**
    1. Creas la ruta: mkdir -p .github/workflows
    2. Creas el archivo YAML: touch .github/workflows/deploy.yml
    3. Contenido básico del archivo YAML:
    4. YAML

```
name: Mi Workflow de Examen
on: [push] # Se dispara cada vez que haces push
jobs:
```

```
tarea:
  runs-on: ubuntu-latest
  steps:
    - name: Checkout del código
      uses: actions/checkout@v3
    - name: Imprimir mensaje
      run: echo "¡El robot de GitHub ha ejecutado esto solo!"
```

5.
6.
7. Haces git add, git commit y git push de este archivo para que empiece a funcionar en la pestaña *Actions* de GitHub.

5. Fase de Despliegue (GitHub Pages)

Publicas tu web al mundo entero.

- **Concepto:** Conviertes tus archivos estáticos en una web pública y accesible mediante URL.
- **Cómo se hace:**
  - Vas a tu repositorio en GitHub → **Settings** → **Pages**.
  - Seleccionas la fuente de despliegue (la rama main o mediante un workflow automatizado si tu examen lo pide así).
  - ¡Te da una URL y listo!

Doc1:

SISTEMES DE CONTROL DE VERSIONS

📚 Apuntes: Sistemas de Control de Versiones y Git

1. Conceptos Fundamentales y Terminología

- **Sistema de Control de Versiones (SCV):** Sistema que registra los cambios realizados en uno o varios archivos a lo largo del tiempo para poder recuperar versiones específicas más adelante.
- **Repositorio (Repository):** Lugar donde se guardan los archivos actuales y todo el historial de cambios.
- **Tronco / Rama Principal (*Trunk* / *Master* / *Main*):** Branca principal del repositorio desde la cual se bifurcan las demás.
- **Rama (*Branch*):** Bifurcación independiente del código que permite desarrollar nuevas funcionalidades o corregir errores sin afectar al tronco principal ni a otras ramas.
- **HEAD / Capçalera:** Puntero que referencia a la versión o *commit* más reciente de la rama en la que se está trabajando.
- **Copia de trabajo (*Working copy*):** Copia local de los archivos en la que el usuario realiza modificaciones antes de enviarlas al repositorio.
- **Commit / Check-in:** Guardar localmente una instantánea de los cambios realizados en el repositorio.
- **Checkout:** Cambiar de rama o recuperar una versión específica en la área de trabajo.
- **Pull:** Copia y fusiona los cambios de un repositorio (usualmente remoto) en la copia local.
- **Push:** Envía las confirmaciones (*commits*) del repositorio local al remoto.
- **Fetch:** Descarga los cambios del repositorio remoto al local sin fusionarlos automáticamente con el trabajo actual.
- **Conflict (Conflicto):** Situación que ocurre cuando dos usuarios modifican las mismas líneas de un archivo y el sistema no puede fusionar las diferencias de forma automática.
- **Merge (Fusionar):** Combinar los cambios de dos ramas independientes en una sola.
- **Tag / Etiqueta:** Marca o alias que se asigna a un *commit* concreto para identificar hitos o versiones de lanzamiento (ej. v1.0).
- **Fork:** Creación de una copia independiente de un repositorio en otro servidor/cuenta.
- **Clone:** Descarga de una copia completa e idéntica de un repositorio existente.

3. Arquitectura y Estados en Git

🔄 Los 3 Estados de los Archivos en Git

1. **Modificado (*Modified*):** El archivo ha sido cambiado en el directorio de trabajo pero aún no se ha marcado para guardar.
2. **Preparado (*Staged* / *Staging Area*):** Los archivos modificados se han marcado para ser incluidos en la próxima confirmación.
3. **Confirmado (*Committed*):** Los datos están almacenados de forma segura en la base de datos local de .git.

🛠️ Las 3 Áreas de Trabajo

- **Working Directory:** Directorio local donde se modifican físicamente los archivos.
- **Staging Area (Index):** Zona intermedia donde se agrupan las modificaciones preparadas para el *commit*.
- **Git Directory (.git):** Base de datos que almacena la metainformación y los objetos del repositorio.

4. Comandos Clave de Git

Configuración e Inicialización

- git config --global user.name "Tu Nombre": Configura el nombre del usuario a nivel global.
- git init: Inicializa un nuevo repositorio Git dentro del directorio actual (crea la carpeta oculta .git).
- git clone <URL>: Clona un repositorio remoto completo a tu equipo.

Flujo de Trabajo Básico

- git status: Muestra el estado de los archivos (rastreados, modificados, en staging, etc.).
- git add <archivo> / git add .: Pasa los archivos del directorio de trabajo al *Staging Area*.
- git commit -m "Mensaje del commit": Registra una instantánea de los cambios acumulados en el *Staging Area*.
- git log / git log --oneline: Muestra el historial cronológico de los *commits* realizados.
- git rm <archivo>: Elimina un archivo del área de seguimiento y del disco.

Trabajo con Ramas (*Branches*)

- git branch: Lista las ramas locales disponibles.
- git branch <nombre_rama>: Crea una nueva rama apuntando al *commit* actual.
- git checkout <nombre_rama>: Cambia la copia de trabajo a la rama seleccionada.
- git checkout -b <nueva_rama>: Crea la rama y cambia a ella de forma inmediata.
- git merge <rama>: Unifica la rama especificada dentro de la rama en la que te encuentras posicionado.

Sincronización Remota

- git fetch origin: Descarga las referencias del servidor sin modificar el directorio de trabajo local.
- git pull origin <rama>: Trae los cambios del servidor remoto y los fusiona directamente en la rama activa.
- git push origin <rama>: Sube las confirmaciones locales a la rama remota especificada.

Archivos Inexistentes / Ignorados (.gitignore)

- Permite definir reglas mediante un archivo .gitignore para omitir archivos temporales, binarios o claves privadas que no deben ser rastreados por Git.

5. Estrategia de Ramificación: GitFlow

Es un modelo de ramificación disciplinado que asigna funciones específicas a distintas ramas en el ciclo de vida del proyecto:

- **Ramas Principales (Permanentes):**
  - master / main: Contiene exclusivamente código estable listo para producción.
  - develop: Acumula las últimas características listas para el próximo lanzamiento.

Doc2:

💻 Guía Práctica de Git para Examen (Consola y Comandos)

🚀 1. Configuración e Inicialización

- **Crear un repositorio local:**

git init → *Crea la carpeta oculta .git en el directorio actual para comenzar a rastrearlo*.

📝 2. Estados y Gestión de Archivos en Local

Flujo de trabajo principal:

1. **Ver el estado de los archivos:**

git status → *Indica qué archivos están modificados, en el Staging Area o sin seguimiento (untracked)*.

2. **Añadir archivos al Staging Area (git add):**

```
git add README.md       # Añade un archivo específico
git add .               # Añade todos los archivos modificados/nuevos
```

3. **Confirmar cambios (git commit):**
4. Bash

git commit -m "Mensaje explicativo del cambio"

🔍 3. Historial e Inspección

- **Consultar el historial de commits:**

git log → *Muestra identificador (hash), autor, fecha y mensaje de cada commit*.

🚫 4. Ignorar Archivos (.gitignore)

Un archivo .gitignore permite definir qué archivos o carpetas no deben ser rastreados por Git.

🗑️ 5. Eliminar y Desplazar Archivos (git rm)

- **Eliminar archivo del disco y del seguimiento:**

git rm archivo.txt → *Elimina el archivo del directorio de trabajo y lo coloca directo en el Staging Area*.

🔄 6. Deshacer Cambios (git reset vs git revert)

A) Usando git reset (Reescribe el historial / Mueve el HEAD)

- **Quitar un archivo del Staging Area:**

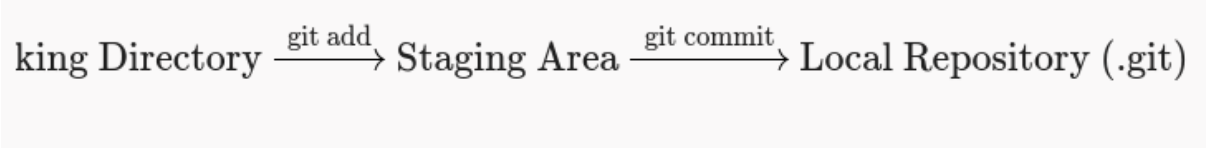
git reset nombreArchivo → *El archivo deja de estar en preparación pero conserva los cambios en el directorio local*.

B) Usando git revert (Opción Segura)

- **Invertir un commit creando uno nuevo:**

git revert <hash_commit> → *No borra el historial existente ni mueve el HEAD. En su lugar, aplica los cambios opuestos y crea un commit nuevo.* **Es la opción más segura**.

📌 Resumen visual del ciclo práctico para el examen



Doc3:

💻 Guía Práctica de Git (Avanzado, Ramas y Etiquetas)

🌐 1. Gestión de Repositorios Remotos (git remote)

- **Clonar un repositorio:**

git clone <URL_remota> → Descarga una copia completa de un repositorio remoto al equipo local.

- **Clonar asignando un nombre personalizado a la carpeta:**

git clone <URL_remota> nombre_carpeta_nueva → El directorio de destino pasará a llamarse como indiques.

🚀 2. Sincronización con el Servidor (git push, git pull, git fetch)

- **Enviar cambios al servidor (git push):**

git push origin main → Sube los cambios confirmados (*commits*) de tu rama local al servidor remoto.

- **Actualizar el repositorio local trayendo y fusionando cambios (git pull):**

git pull origin main → Descarga los cambios del servidor remoto y los fusiona automáticamente en tu rama local. (¡Ojo! Es obligatorio hacer commit de tus cambios locales antes de ejecutar un pull).

- **Descargar información sin fusionar (git fetch):**

git fetch origin → Descarga los datos del servidor remoto y los ubica en una rama oculta local (remotes/origin/main), pero **no** los fusiona automáticamente con tu código actual, permitiéndote inspeccionarlos antes.

🌿 3. Trabajo con Ramas (git branch y git checkout)

- **Consultar ramas:**

```
git branch -a       # Muestra todas las ramas (locales y remotas)
```

- **Crear una nueva rama:**

git branch nom_nova_rama → Crea el puntero local pero **no** te posiciona en ella de manera automática

- **Desplazarse entre ramas (git checkout):**

git checkout nom_branca → Cambia la copia de trabajo a la rama seleccionada.

- **Crear una rama y cambiar a ella en un solo paso:**

git checkout -b nom_branca → Crea la nueva rama local y sitúa el puntero de HEAD en ella de forma inmediata.

- **Subir una rama local al servidor remoto:**

git push -u origin nom_branca_creada

- **Eliminar ramas:**

```
git branch -d nom_branca_creada              # Elimina la rama localmente
git push origin --delete nom_branca_creada   # Elimina la rama del servidor remoto
```

🔀 4. Fusión de Ramas (git merge)

- **Fusionar una rama en la actual:**

git merge nom_rama_a_fusionar → Toma las líneas independientes de desarrollo y las integra en la rama receptora en la que te encuentres.

- ⚠️ **Pasos previos obligatorios antes de un git merge para evitar problemas:**
  1. Asegurarte de estar posicionado en la rama receptora (comprobar con git status o cambiar con git checkout).
  2. Actualizar tanto la rama receptora como la que vas a fusionar usando git pull o git fetch + git merge para tener los últimos cambios remotos.

🏷️ 5. Etiquetado (git tag)

Las etiquetas se utilizan para marcar puntos importantes del historial, como versiones de lanzamiento (v1.0).

- **Listar etiquetas:**

```
git tag                 # Lista todas las etiquetas por orden alfabético
```

- **Crear etiquetas:**

```
git tag v1.0                          # Crea una etiqueta ligera (puntero simple)
```

- **Compartir etiquetas con el servidor remoto:**

```
git push origin nom_etiqueta       # Sube una etiqueta específica
git push origin --tags             # Sube todas las etiquetas locales que no existan en el remoto
```

- **Eliminar etiquetas:**

```
git tag -d nom_tag                 # Elimina la etiqueta localmente
git push origin :refs/tags/nom_tag  # Elimina la etiqueta del servidor remoto
```

- **Trabajar sobre una versión etiquetada:**

```
git checkout -b <nom_nova_branca> nom_etiqueta
```

Como no se puede hacer un checkout directo a una etiqueta para modificarla, se crea una nueva rama a partir de esa etiqueta.

Doc4:

💻 Guía Práctica: GitHub Pages, Colaboración y Buenas Prácticas

🌐 1. GitHub Pages (Publicación de Sitios Web)

GitHub Pages permite alojar sitios web estáticos (con archivos README.md o HTML) directamente desde un repositorio de GitHub.

- Pasos principales para configurarlo:
    1. Crear un repositorio en GitHub.
    2. Subir los archivos del sitio web o documentación (se suele usar una rama dedicada llamada gh-pages o la rama principal).
    3. Ir a **Settings** (Configuración) del repositorio en la web de GitHub.
    4. Buscar la sección de **GitHub Pages**, seleccionar la rama de origen (*Source*) y la carpeta (por ejemplo, /(root)) y guardar.

🤝 2. Colaboración en Proyectos (Fork y Pull Request)

A) Hacer un Fork (Copiar un repositorio)

Hacer un *Fork* significa clonar un repositorio ajeno a tu propia cuenta de GitHub para poder experimentar e introducir cambios libremente sin alterar el proyecto original.

- **Vincular el repositorio original como referencia (*upstream*):** Una vez clonado tu *fork* en local, es una buena práctica añadir la ruta del repositorio original con el alias upstream:

git remote add upstream direccion_repositorio_original

- Actualizar tu repositorio con los cambios del original:

git fetch upstream
git checkout master
git merge upstream

B) Hacer un Pull Request (Proponer cambios)

Una vez modificada tu versión copiada (*fork*), puedes solicitar al autor original que integre tus cambios:

1. Vas a la web del repositorio original y haces clic en **New pull request**.

2. Configuras el *base repository* (el proyecto original y su rama, ej. master) y el *head repository* (tu *fork* y tu rama modificada, ej. ramaEjemplo).
3. Rellenas el formulario explicando los cambios y haces clic en **Create pull request**.
4. Si los mantenedores lo aprueban y fusionan, aparecerá como *Merged* y los cambios se aplicarán con éxito al proyecto principal.

Doc5:

💻 Guía Práctica: GitHub Actions y CI/CD

🔄 1. Conceptos de CI/CD (Integración y Despliegue Continuo)

- **Continuous Integration (CI - Integración Continua):** Práctica donde los desarrolladores integran código en un repositorio compartidos varias veces al día. Cada integración se verifica con tests automatizados para detectar errores de forma temprana.
- **Continuous Delivery (CD - Entrega Continua):** Extensión de la CI que asegura que el código siempre esté listo para ser liberado a producción. Pasa por pruebas automatizadas, dejando el sistema preparado para desplegarse en cualquier momento.
- **Continuous Deployment (CD - Despliegue Continuo):** Práctica avanzada donde cada cambio aprobado y verificado de forma automática se despliega directamente a producción **sin intervención humana**.

⚙️ 2. Componentes de GitHub Actions

Un flujo de trabajo se estructura mediante los siguientes elementos:

- **Workflow (Flujo de trabajo):** Procedimiento automatizado configurable que se añade al repositorio. Se compone de uno o más *jobs* y se activa mediante eventos (como un *push* o *pull request*).
- **Jobs (Trabajos):** Conjunto de pasos (*steps*) que se ejecutan en el mismo *runner*. Por defecto, los *jobs* de un workflow se ejecutan en **paralelo**.
- **Steps (Pasos):** Tareas individuales que se ejecutan secuencialmente dentro de un *job*. Pueden ser una *action* o un comando de shell.
- **Actions (Acciones):** Comandos independientes y reutilizables que se combinan para formar los *steps*. Son el bloque de construcción más pequeño.
- **Runner:** Servidor virtual o local que tiene instalada la aplicación de GitHub Actions y se encarga de ejecutar los *jobs*.

📄 3. Estructura y Sintaxis de un Workflow (YAML)

Los archivos de configuración de GitHub Actions se escriben en formato **YAML** y deben almacenarse obligatoriamente dentro del directorio .github/workflows/.

Ejemplo básico de Workflow:

```yaml
name: primer-ejemplo                    # Nombre opcional del workflow
on: [push]                              # Evento que activa el flujo

jobs:
  check-bats-version:
    runs-on: ubuntu-latest              # Entorno del runner
    steps:
      - uses: actions/checkout@v2       # Descarga el código del repositorio
      - uses: actions/setup-node@v1     # Configura Node.js
      - run: npm install -g bats        # Ejecuta comandos de shell
      - run: bats -v                    # Ejecuta comandos de shell
```

🛠️ 4. Elementos Clave para Examen (Sintaxis YAML)

- **name**: Especifica el nombre del workflow que aparecerá en la pestaña de GitHub Actions.
- **on**: Define los eventos que activan el flujo (ej. push, pull_request, o filtrado por ramas y etiquetas):

```yaml
on:
  push:
    branches: [ main ]
    tags: [ v1 ]
```

- **jobs.<job_id>.needs**: Permite gestionar dependencias entre *jobs* para evitar que se ejecuten en paralelo o condicionar su ejecución:

```yaml
jobs:
  job1:
    # ...
  job2:
    needs: job1         # job2 espera a que job1 termine con éxito
  job3:
    if: always()
    needs: [job1, job2] # job3 se ejecuta siempre que terminen job1 y job2[
```

- **runs-on**: Define el sistema operativo de la máquina virtual (ej. ubuntu-latest, windows-latest, macos-latest).
- **env**: Permite definir variables de entorno a nivel global, de job o de step.
- **uses vs run**:
  - uses: Se utiliza para invocar **Actions** predefinidas (de GitHub o de terceros, ej. actions/checkout@v2).

    - run: Se utiliza para ejecutar **comandos de terminal o scripts de shell** directamente[cite: 125, 134].

Doc6:

💻 Guía Práctica: Custom GitHub Actions (Acciones Personalizadas)

🛠️ 1. Tipos de Custom Actions

Las acciones personalizadas permiten empaquetar tareas para reutilizarlas en flujos de trabajo. Existen dos tipos principales:

1. **Docker-container actions:**
    - El código se empaqueta dentro de un contenedor Docker.
    - Más consistente y fiable, ya que no hay que preocuparse por las dependencias del sistema.
    - *Inconvenientes:* Solo se pueden ejecutar en entornos Linux y son más lentas.
2. **JavaScript actions:**
    - El código se ejecuta directamente sobre el *runner* utilizando Node.js.
    - Su ejecución es mucho más rápida. Debe estar implementada en JavaScript puro para garantizar su compatibilidad.

📄 2. El Archivo de Metadatos (action.yml)

Toda acción personalizada requiere obligatoriamente un archivo de configuración de metadatos llamado action.yml (o action.yaml) en la raíz de su carpeta.

Principales campos de configuración

:

- **name**: Nombre de la acción.
- **description**: Breve descripción de lo que hace.
- **inputs**: Parámetros de entrada que recibe la acción.
    - *Nota práctica:* GitHub convierte automáticamente cada input en una variable de entorno con el formato INPUT_<VARIABLE_NAME> (en mayúsculas y reemplazando espacios por guiones bajos _).
- **outputs**: Datos que la acción devuelve o establece tras su ejecución.
- **runs**: Define cómo se ejecuta la acción (según sea JavaScript o Docker):
    - Para **JavaScript** especifica el archivo principal (main: 'index.js' o compilado en dist/index.js).
    - Para **Docker** especifica el archivo image: 'Dockerfile'.

💻 3. Ejemplo Práctico: JavaScript Action

Pasos para crearla e implementarla:

1. **Crear el archivo de metadatos** (action.yml):
2. YAML

```
name: 'Hello World'

description: 'Greet someone and record the time'

inputs:

  who-to-greet:

    description: 'Who to greet'

    required: true

    default: 'World'

outputs:

  time:

    description: 'The time we greeted you'

runs:

  using: 'node12'

  main: 'dist/index.js'
```

3.
4. **Inicializar Node.js e instalar dependencias oficiales de GitHub Actions**:
5. Bash

```
npm init -y

npm install @actions/core

npm install @actions/github
```

6.
7. **Escribir la lógica principal** (index.js) utilizando @actions/core para recoger inputs y devolver outputs:
8. JavaScript

```
const core = require('@actions/core');

const github = require('@actions/github');
```

```
try {
    const nameToGreet = core.getInput('who-to-greet');
    console.log(`Hello ${nameToGreet}!`);
    const time = (new Date()).toTimeString();
    core.setOutput("time", time);
} catch (error) {
    core.setFailed(error.message);
}
```

9.
10. **Compilar el proyecto** (para evitar subir la carpeta node_modules al repositorio) utilizando ncc:
11. Bash

```
npm i -g @vercel/ncc
ncc build index.js --license licenses.txt
```

12.
13. (Asegúrate de que el campo main en action.yml apunte al archivo compilado, ej: dist/index.js).
14. **Probar la acción local dentro de un Workflow** (.github/workflows/workflow.yml):
15. YAML

```
on: [push]
jobs:
  hello_world_job:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v2
      - name: Hello world action step
        uses: ./path/to/action/folder    # Ruta local de tu action
        id: hello
```

```
        with:
          who-to-greet: 'Mona the Octocat'
      - name: Get the output time
        run: echo "The time was ${{ steps.hello.outputs.time }}"
```

16.

🐳 4. Ejemplo Práctico: Docker Action

1. **Crear el Dockerfile**:
2. Dockerfile

```
FROM alpine:3.10

COPY entrypoint.sh /entrypoint.sh

ENTRYPOINT ["/entrypoint.sh"]
```

3.
4. **Crear el script de entrada** (entrypoint.sh):
5. Bash

```
#!/bin/sh -l

echo "Hello $1"

time=$(date)

echo "::set-output name=time::$time"
```

6.
7. **Configurar el action.yml para Docker**:
8. YAML

```
name: 'Hello World'

description: 'Greet someone'

inputs:

  who-to-greet:

    required: true

    default: 'World'

runs:

  using: 'docker'
```

```
image: 'Dockerfile'
args:
  - ${{ inputs.who-to-greet }}
```

doc7:
Aquí tienes la **Guía Práctica Resumen (Cheat Sheet) de GitHub Actions** basada en el último documento, explicada en castellano y orientada a los conceptos y sintaxis clave para el examen:

💻 Guía Práctica: GitHub Actions Cheat Sheet

📄 1. Estructura y Sintaxis del Workflow

- **Ubicación y extensión:** Los archivos de flujo de trabajo deben escribirse en sintaxis YAML (.yml o .yaml) y guardarse obligatoriamente en el directorio .github/workflows/ de tu repositorio. Cada archivo YAML corresponde a un workflow independiente.
- **Nombre del Workflow (name):** Es el nombre descriptivo que aparecerá en la pestaña de actions de tu repositorio.
- **Variables de entorno (env):** Se pueden definir mapas de variables de entorno a nivel de workflow, job o step. Existen variables predeterminadas por defecto como GITHUB_SHA, GITHUB_REF, GITHUB_EVENT_NAME, HOME, etc. Además, se pueden utilizar secretos mediante el contexto secrets.GITHUB_TOKEN para llamadas a la API o comandos de Git.

⚡ 2. Disparadores (on Event)

El bloque on define qué evento activa la ejecución del workflow. Puede ser una cadena de texto, un array de eventos o un mapa de configuración detallado:

- **Filtros de ramas y etiquetas (branches / tags):** En eventos como push o pull_request, permiten seleccionar o excluir (usando el prefijo ! o -ignore como branches-ignore) las referencias de git que activarán el flujo.
- **Filtros de rutas (paths):** Especifican qué archivos exactos deben haber sido modificados para que el workflow se ejecute.
- **Actividades específicas (types):** Acotan la ejecución a acciones concretas (como opened, created, edited, etc.).
- **Ejecución programada (schedule):** Permite lanzar workflows de forma periódica mediante expresiones cron (ej: cron: '*/15 * * * *').

🏗️ 3. Trabajos (jobs Collection)

Un workflow se compone de uno o más jobs identificados por un job_id único.

- **Ejecución:** Por defecto, los jobs se ejecutan en **paralelo** a menos que se indique lo contrario. Cada job corre en una instancia limpia y nueva del entorno virtual especificado.
- **needs:** Identifica qué job o jobs deben completarse con éxito antes de que el actual pueda iniciarse. Si un job de dependencia falla, los jobs que lo necesitan se omiten automáticamente (a menos que usen condicionales).

- **runs-on:** Define el tipo de máquina virtual anfitriona (ej. ubuntu-latest, windows-latest, macos-latest, versiones específicas como ubuntu-20.04, o ejecutores propios mediante etiquetas como [self-hosted, linux]).
- **container y services:** Permiten ejecutar los pasos dentro de contenedores Docker específicos o levantar servicios adicionales de apoyo (como bases de datos o servidores Redis) de forma automatizada.
- **timeout-minutes:** Establece el tiempo máximo de ejecución en minutos antes de que GitHub cancele el workflow automáticamente (por defecto: 360).

🚶 4. Pasos del Job (steps)

Cada job contiene una secuencia secuencial de tareas llamadas *steps*. Cada paso se ejecuta en su propio proceso dentro del entorno virtual y tiene acceso al espacio de trabajo.

- **name:** Etiqueta opcional pero muy recomendada para mejorar la legibilidad de los logs.
- **uses:** Ejecuta una **Action** existente. Se recomienda encarecidamente indicar siempre la versión mediante una referencia de Git, rama, SHA o etiqueta:
  - uses: {owner}/{repo}@{ref} para acciones en repositorios públicos.
  - uses: {owner}/{repo}/{path}@{ref} para acciones en un subdirectorio.
  - uses: ./path/to/dir para acciones locales en el mismo repositorio.
  - uses: docker://{image}:{tag} para imágenes alojadas en Docker Hub o registros públicos.
- **with:** Mapa de parámetros de entrada definidos en el archivo action.yml de la acción. Si la acción está basada en contenedores, se usan parámetros especiales como args (para pasar inputs al ENTRYPOINT) o entrypoint (para sobrescribirlo).
- **run:** Ejecuta un **programa de línea de comandos** utilizando la shell del sistema operativo. Cada palabra clave run abre un proceso nuevo, y se pueden encadenar varios comandos usando el operador de tubería (|).
- **if:** Permite aplicar una condición lógica sin necesidad de usar bloques ${{ }} para decidir si un paso debe ejecutarse o no.

📊 5. Estrategias, Contextos y Almacenamiento

- **Estrategia de Matriz (strategy.matrix):** Permite ejecutar el conjunto de pasos de un job sobre múltiples configuraciones combinadas (por ejemplo, probando varios sistemas operativos o versiones de lenguajes en paralelo). El parámetro fail-fast (por defecto true) cancela todos los jobs en curso si uno de ellos falla.
- **Contextos y Expresiones (${{ }}):** Salvo en las condiciones if, las expresiones se escriben siempre dentro de un bloque ${{ }} para evaluar operadores, funciones o acceder a objetos de contexto disponibles como github, job, steps, runner, secrets, strategy y matrix.
- **Almacenamiento de Artefactos y Caché:**
  - **Artefactos:** Ficheros generados durante la ejecución que se pueden compartir entre jobs usando las acciones oficiales actions/upload-artifact y actions/download-artifact (disponibles para descarga web durante 90 días).
  - **Caché:** Para reutilizar dependencias entre ejecuciones (como node_modules), se utiliza la acción actions/cache configurando rutas (path),

claves de búsqueda (key) y claves alternativas de restauración (restore-keys).